

第7讲 · 练习题与答案

从 RNN、LSTM 到 Attention: Transformer 为什么会出现？

使用方法

建议先只打印/查看前面的题目页，独立写答案后再看解析。能够用自己的话画出“RNN $\rightarrow$ LSTM $\rightarrow$ Attention $\rightarrow$ Transformer”的因果链，比背定义更重要。

1. 为什么语言模型不能把每个 token 完全独立处理？
2. RNN 的隐藏状态 h_t 起什么作用？
3. 为什么位置 1 的信息要影响位置 100 时，普通 RNN 会比较困难？
4. 梯度消失是什么？它为什么会影响长期依赖学习？
5. LSTM 的遗忘门、输入门、输出门分别解决什么直觉问题？

6. LSTM 已经缓解长期依赖，为什么后来还需要 Attention?
7. 早期 Seq2Seq 的固定 Context 为什么可能成为瓶颈?
8. 用一句话解释 Attention 的核心思想。
9. Attention 为什么能缩短远距离信息的访问路径?
10. Self-Attention 和 Encoder-Decoder Attention 的“谁看谁”有什么区别?

11. Transformer 为什么更容易利用 GPU 并行?
12. Transformer 是否“完全不需要顺序信息”? 为什么?
13. 标准 Self-Attention 在很长序列上会带来什么新问题?
14. 请画出你自己的技术演化链: $RNN \rightarrow LSTM \rightarrow Attention \rightarrow Transformer$, 并为每一步写一个“它主要解决了什么”。
15. 结合 Agent Systems: 为什么一个 Agent 的上下文越来越长会成为系统问题?

参考答案

1. 因为语言含义依赖上下文和词序；同一个 token 在不同上下文可能表示不同含义。
2. h_t 是读到当前位置后的内部状态，它把前面已经处理的信息带到下一时间步。
3. 信息需要经过许多次 h 的递推，表示可能被覆盖；反向传播也需要穿过很长计算链。
4. 梯度在链式传播中越来越小，早期位置几乎收不到学习信号，因而难以学到远距离依赖。
5. 遗忘门决定旧信息保留多少；输入门决定新信息写多少；输出门决定当前状态输出多少。

6. LSTM 仍按时间步顺序递推，难完全并行；长序列的信息访问仍需要经过递归链。
7. 整句所有信息被压缩成一个固定长度向量，句子越长越容易丢失细节。
8. 当前计算根据相关性给多个位置分配权重，再按权重读取和汇总这些位置的信息。
9. 目标位置可以直接访问远处位置，不必经由每一个中间时间步逐级传递。
10. Encoder-Decoder Attention 是 Decoder 看 Encoder；Self-Attention 是同一序列内部各位置彼此看。

11. 一层内各 token 的矩阵计算可以批量执行，不必等待前一个时间步先完成。

12. 不是。Transformer 去掉了递归，所以需要位置编码等机制把顺序/位置提供给模型。

13. 标准实现的注意力关系数随 n 增长，带来计算、显存和带宽压力。

14.

RNN: 引入序列状态; LSTM: 改善长期记忆/梯度; Attention: 动态直接读取相关位置; Transformer: 以 Self-Attention 为主干并提高并行能力。

15. 更长上下文意味着更多 token 参与 Attention、Prefill

和缓存，增加延迟、算力和内存，同时模型也未必能有效利用全部信息。